

keyle

v0.3.0

2026-06-01

MIT

This package provides a simple way to style keyboard shortcuts in your documentation.

MAGICWENLI

✉ yxnian@outlook.com

Table of Contents

About	2
Usage	3
II.1 Importing	3
II.2 Quick Start	3
Themes	4
III.1 Built-in Themes	4
III.2 Custom Themes	4
III.3 Extending a Theme with <code>.with()</code> ..	5
Symbols	6
IV.1 Mac Keyboard Symbols	6
IV.2 Linux Biolinum Keyboard Symbols .	7
IV.3 SVG Key Glyphs	9
Available commands	10
Index	15

Part I

About

KEYLE is a library that allows you to create HTML `<kbd>` like keyboard shortcuts simple and easy.

The name, keyle, is a combination of key and theme.

This project is inspired by [auth0/kbd](http://github.com/auth0/kbd)¹ and [dogezzen/badgery](https://github.com/dogezzen/badgery)². Send them respect!

¹<http://github.com/auth0/kbd>

²<https://github.com/dogezzen/badgery>

Part II

Usage

II.1 Importing

`KEYLE` is imported using

```
#import "@preview/keyle:0.3.0"
```

II.2 Quick Start

Generate a keyboard renderer with `#config` function. Section [Section III.1](#) lists available themes.

```
#let kbd = keyle.config()
#kbd("Ctrl", "Shift", "Alt", "Del")

#let kbd = keyle.config(
  theme: keyle.themes.bioluminum,
  delim: keyle.bioluminum-key.delim_plus
)
#kbd("Ctrl", "Shift", "Alt", "Del")
```

Ctrl + Shift + Alt + Del

Ctrl + ⇧ + Alt + Del

There are `compact` and `delim` options to make the output more compact and change the delimiter between keys.

You can either use them in `#config` function or directly in generated `kbd` function.

```
#let kbd = keyle.config(compact: true, delim: "-")
#kbd("Ctrl", "Shift", "Alt", "Del")

#let kbd = keyle.config()
#kbd("Ctrl", "Shift", "Alt", "Del", compact: true, delim: "-")
```

Ctrl-Shift-Alt-Del

Ctrl-Shift-Alt-Del

Part III

Themes

III.1 Built-in Themes

Themes function are available at `#keyle.themes` dictionary.

`#keyle.themes.standard`

Home

`#keyle.themes.deep-blue`

HOME

`#keyle.themes.type-writer`

HOME

`#keyle.themes.minimal`

Home

`#keyle.themes.radix`

Home

`#keyle.themes.flowbite`

Home

`#keyle.themes.flowbite-dark`

Home

`#keyle.themes.daisy`

Home

`#keyle.themes.bioluminum`

Home

III.2 Custom Themes

You can create your own theme by defining a function that takes a string and returns a themed content.

```
// https://www.radix-ui.com/themes/playground#kbd
#let radix_kdb(content) = box(
  rect(
    inset: (x: 0.5em),
    outset: (y: 0.05em),
    stroke: rgb("#1c2024") + 0.3pt,
    radius: 0.35em,
    fill: rgb("#fcfcfd"),
    text(fill: black, font: (
      "Helvetica Neue",
      "TeX Gyre Heros",
    ), content),
  ),
)
#let kbd = keyle.config(theme: radix_kdb)
#kbd("⌘ D") #kbd("^ F")
```



III.3 Extending a Theme with .with()

Every built-in theme is a `#keycap` or `#svg-keycap` factory with its style pre-bound. Because they are ordinary functions, you extend any of them using Typst's native `.with(...)`, without learning a bespoke override API. The text layer (text-args, wrap) and the cap layer (fill, stroke, radius, raise, ...) stay separate.

```
#let rose = keyle.themes.flowbite.with(
  fill: rgb("#fee2e2"),
  stroke: rgb("#fca5a5"),
  text-args: (fill: rgb("#991b1b"), weight: "bold"),
)
#let kbd = keyle.config(theme: rose)
#kbd("Ctrl", "Shift", "K")
```



Part IV

Symbols

IV.1 Mac Keyboard Symbols

KEYLE provides symbols for Mac keyboard. You may need install Fira Code font to see the symbols correctly.

- [Fira Code @ Google Fonts](https://fonts.google.com/specimen/Fira+Code?preview.text=%E2%8C%98%E2%87%A7%E2%8C%A5%E2%8C%83%E2%86%A9&query=fira+code)³
- [Apple Mac Keyboard Symbols](https://support.apple.com/en-hk/guide/mac-help/cpmh0011/mac)⁴

³<https://fonts.google.com/specimen/Fira+Code?preview.text=%E2%8C%98%E2%87%A7%E2%8C%A5%E2%8C%83%E2%86%A9&query=fira+code>

⁴<https://support.apple.com/en-hk/guide/mac-help/cpmh0011/mac>

```
#let mac-key = keyle.mac-key
#let kbd = keyle.config(theme: keyle.themes.standard)
#let mac-key-font = (
  "FiraCode Nerd Font Mono"
)
#grid(
  columns: (2fr, 1fr, 2fr, 1fr),
  rows: 2em,
  align: horizon,
  ..mac-key.pairs().map(item => (
    raw("#mac-key." + item.at(0)),
    kbd(
      text(font: mac-key-font, item.at(1))
    ),
  )),flatten()
)
```

#mac-key.command		#mac-key.shift	
#mac-key.option		#mac-key.control	
#mac-key.return		#mac-key.delete	
#mac-key.forward-delete		#mac-key.escape	
#mac-key.left		#mac-key.right	
#mac-key.up		#mac-key.down	
#mac-key.pageup		#mac-key.pagedown	
#mac-key.home		#mac-key.end	
#mac-key.tab-right		#mac-key.tab-left	

IV.2 Linux Biolinum Keyboard Symbols

KEYLE provides symbols for Linux Biolinum Keyboard font.

- [Linux Biolinum Keyboard⁵](https://libertine-fonts.org/)

⁵<https://libertine-fonts.org/>

```
#let biolinum-key = keyle.biolinum-key
#let kbd = keyle.config(theme: keyle.themes.biolinum)
#grid(
  columns: (5fr, 3fr, 5fr, 3fr),
  rows: 2em,
  align: horizon,
  ..biolinum-key.pairs().map(item => (
    raw("#biolinum-key." + item.at(0)),
    kbd(item.at(1)),
  )).flatten()
)
```

#biolinum-key.Strg		#biolinum-key.Alt	
#biolinum-key.Ctrl		#biolinum-key.Shift	
#biolinum-key.Tab		#biolinum-key.Enter	
#biolinum-key.Capslock		#biolinum-key.Home	
#biolinum-key.Del		#biolinum-key.Ins	
#biolinum-key.End		#biolinum-key.Space	
#biolinum-key.Esc		#biolinum-key.PageUp	
#biolinum-key.PageDown		#biolinum-key.Back	
#biolinum-key.Pad_0		#biolinum-key.Pad_1	
#biolinum-key.Pad_2		#biolinum-key.Pad_3	
#biolinum-key.Pad_4		#biolinum-key.Pad_5	
#biolinum-key.Pad_6		#biolinum-key.Pad_7	
#biolinum-key.Pad_8		#biolinum-key.Pad_9	
#biolinum-key.Pad_Div		#biolinum-key.Pad_Add	
#biolinum-key.Pad_Sub		#biolinum-key.Pad_Mul	
#biolinum-key.Pad_Enter		#biolinum-key.Mac_Cmd	
#biolinum-key.Mac_Opt		#biolinum-key.Mac_FDel	
#biolinum-key.Mac_Del		#biolinum-key.GNU	
#biolinum-key.Win		#biolinum-key.Tux	
#biolinum-key.delim_plus+		#biolinum-key.delim_minus-	-

IV.3 SVG Key Glyphs

A key symbol is just content, so non-textual keys can be passed as inline SVG glyphs from `#keyle.svg-key`. Available names are up, down, left, right, enter, backspace, tab and win.

```
#let kbd = keyle.config(theme: keyle.themes.flowbite)
#kbd(keyle.svg-key.up) #kbd(keyle.svg-key.down)
#kbd(keyle.svg-key.left) #kbd(keyle.svg-key.right)
#kbd(keyle.svg-key.enter) #kbd(keyle.svg-key.backspace)
#kbd(keyle.svg-key.tab) #kbd(keyle.svg-key.win)
```



Part V

Available commands

[#config](#)

[#gen-examples](#)

[#join-keys](#)

#config(**{theme}**): **themes.standard**, **{compact}**: **false**, **{delim}**: **"+"**) → **function**

Config function to generate keyboard rendering helper function.

Argument

{theme}: **themes.standard**

function

The theme function to use. Any preset from themes, optionally extended with `.with(...)`, or a custom `sym => content` function.

Argument

{compact}: **false**

bool

Whether to render keys in a compact format.

Argument

{delim}: **"+"**

str

The delimiter to use between keys.

#gen-examples(**{kbd}**) → **content**

Generate examples for the given keyboard rendering function.

Argument

{kbd}

function

The keyboard rendering function.

#join-keys(**{keys}**, **{theme}**, **{delim}**) → **content**

Join rendered keys with a delimiter between them.

[#_len](#)

[#keycap](#)

[#svg-keycap](#)

[#_rect](#)

[#style-text](#)

#_len(**{v}**) → **length**

Coerce a number (treated as points) or a length into a length.

```
#_rect(
```

```
  {x},
  {y},
  {w},
  {h},
  {r},
  {fill},
  {stroke}: none,
  {sw}: 0
```

```
) → bytes
```

Build a parameterized keycap SVG (pure vector, no <foreignObject>). All sizes are in points; colors are CSS hex strings or none.

```
#keycap(
```

```
  {fill}: rgb("#eeeeee"),
  {stroke}: rgb("#555555") + 0.6pt,
  {radius}: 2pt,
  {inset}: (x: 3pt, y: 2.5pt),
  {raise}: 1.2pt,
  {shadow}: rgb("#555555"),
  {baseline}: 0.1em,
  {text-args}: (fill: black),
  {wrap}: it => it,
  {sym})
```

```
) → content
```

Keycap factory using vector rectangles (the rect backend).

Bind styling with `.with(...)` to obtain a theme renderer `sym => content`.

Argument

```
{fill}: rgb("#eeeeee")
```

color

Face fill color.

Argument

```
{stroke}: rgb("#555555") + 0.6pt
```

stroke

Face border.

Argument

```
{radius}: 2pt
```

length | ratio

Corner radius.

Argument

```
{inset}: (x: 3pt, y: 2.5pt)
```

length | dictionary

Inner padding around the symbol.

V Available commands

Argument	
<code>(raise): 1.2pt</code>	length
Diagonal drop-shadow extent; 0pt renders a flat cap.	
Argument	
<code>(shadow): rgb("#555555")</code>	color none
Shadow color, or none to disable.	
Argument	
<code>(baseline): 0.1em</code>	length ratio
Baseline shift of the cap relative to surrounding text.	
Argument	
<code>(text-args): (fill: black)</code>	dictionary
Text layer arguments spread into text.	
Argument	
<code>(wrap): it => it</code>	function
Text layer content transform.	
Argument	
<code>(sym)</code>	any
The key symbol.	

#style-text(`(sym)`, `(args): (:)`, `(wrap): it => it`) → **content**

Text layer: turn a key symbol into styled content.

wrap may be any content -> content function, so builtin transforms can be passed directly, e.g. wrap: smallcaps or wrap: upper.

Argument	
<code>(sym)</code>	any
The key symbol.	
Argument	
<code>(args): (:)</code>	dictionary
Arguments spread into text.	
Argument	
<code>(wrap): it => it</code>	function
Content transform applied after styling.	

```
#svg-keycap(
  {fill}: rgb("#f3f4f6"),
  {stroke}: rgb("#d1d5db"),
  {stroke-width}: 1.2,
  {radius}: 6,
  {lip}: 3,
  {shadow}: auto,
  {pad}: (x: 6pt, y: 3.5pt),
  {baseline}: 0.3em,
  {text-args}: (fill: rgb("#1f2937"), weight: "semibold"),
  {wrap}: it => it,
  {sym})
) → content
```

Keycap factory using a generated SVG shell (the svg backend).

The SVG is stretched to the measured symbol size so corners never distort. Bind styling with `.with(...)` to obtain a theme renderer `sym => content`.

Argument	
{fill}: rgb("#f3f4f6")	color
Face fill color.	
Argument	
{stroke}: rgb("#d1d5db")	color
Face border color.	
Argument	
{stroke-width}: 1.2	float
Border width in points.	
Argument	
{radius}: 6	float
Corner radius in points.	
Argument	
{lip}: 3	float
Bottom shadow lip height in points.	
Argument	
{shadow}: auto	color auto none
Shadow color, auto to derive from stroke, or none.	
Argument	
{pad}: (x: 6pt, y: 3.5pt)	dictionary

V Available commands

Inner padding around the symbol.

Argument

`<baseline>: 0.3em`

`length` | `ratio`

Baseline shift of the cap relative to surrounding text.

Argument

`<text-args>: (fill: rgb("#1f2937"), weight: "semibold")`

`dictionary`

Text layer arguments spread into text.

Argument

`<wrap>: it => it`

`function`

Text layer content transform.

Argument

`<sym>`

`any`

The key symbol.

Part VI

Index

C

`#config` 3, 10

G

`#gen-examples` 10

J

`#join-keys` 10

K

`#keycap` 5, 10, 11

S

`#style-text` 10, 12

`#svg-keycap` 5, 10, 13

—

`#_len` 10

`#_rect` 10, 11